

COS30049 ASSIGNMENT 3

Group 5 - May 2025

Group members:

Hoang Minh Duc (105541452),
Dang Giang Son (104977506),
Nguyen Quy Khanh (104189712)

Convener, Lecturer: Dr. Le Minh Duc

Table of contents

PROJECT INFORMATION	3
1. Project Name:	3
2. Project Description:	4
3. Team Information	4
4. Version Information	4
EXECUTIVE SUMMARY	4
ABI ILLUSTRATION	5
Errors:	5
OwnableUnauthorizedAccount	5
Events	5
OwnershipTransferred	5
Paused	6
ProductCreated	6
ProductDeactivated	7
ProductPurchased	8
Unpaused	9
Functions	9
createProduct	9
buyProduct	10
getPurchases	10
pause	11
unpause	12
deactivateProduct	12
transferOwnership	12
Call Graph Discussion	13
Overview:	13
Inheritance graph discussion	15
Inheritance relationships:	15
Code Reusability:	15
Function and State Variable Overriding:	16
Function Visibility and Access Control	16
Modifier Application	16
Conclusion	16
References:	16

PROJECT INFORMATION

1. Project Name:

Happy Farming

2. Project Description:

Our project, “Happy Farming”, is a blockchain-based project allowing the exchange of agricultural products via blockchain technology. We implement modern technology combined with user-friendly UI, to create a decentralised market platform for farmers to upload and sell their agricultural products.

3. Team Information

- Hoang Minh Duc: Team leader, handles the central parts of the frontend and most of the backend operations.
- Dang Giang Son: Main author of blockchain-related development (wallet integration, writing smart contracts, etc)
- Nguyen Quy Khanh: Helps finalise the code, writes reports, and handles most of the frontend development.

4. Version Information

Smart contract version: Solidity Version: ^0.8.20

The ABI is compiled using Hardhat and utilises the OpenZeppelin library version 4.9.6 in the smart contracts.

EXECUTIVE SUMMARY

This report analyses the *Marketplace* smart contract, designed as a decentralised platform for product listing and purchase. Developed in Solidity and leveraging OpenZeppelin libraries, the contract ensures modularity, access control, and protection against common vulnerabilities.

Key tools and methods:

Solidity Visual Developer (VS Code extension) for ABI, inheritance, and call graph visualisation.

Core functionalities:

- Product lifecycle management (createProduct, deactivateProduct)

- Secure transaction processing (buyProduct)
- User-centric data access (getPurchases)
- Owner-only administrative controls (pause, unpause, transferOwnership)

Security is enforced via modifiers

- onlyOwner for restricted actions
- whenNotPaused and nonReentrant for operational integrity

The contract meets the essential criteria for a simple NFT-style marketplace, including secure interactions, reusable components, and clearly divided logic.

ABI ILLUSTRATION

The provided ABI (Application Binary Interface) contains:

2 errors, 6 events, 15 functions with 9 functions non-view (state-changing) and 6 functions view (read-only)

Errors:

OwnableUnauthorizedAccount(address account)

```
24 {
25   "inputs": [
26     {
27       "internalType": "address",
28       "name": "account",
29       "type": "address"
30     }
31   ],
32   "name": "OwnableUnauthorizedAccount",
33   "type": "error"
34 },
```

You, 20 hours ago • Uncommitted changes

Parameter:

- **account** (type address):

Functionality:

- Giving an error notification when an unauthorised address attempts to perform restricted operations.

Security Impact:

- Prevents unauthorised access to critical administrative functions, enhancing access control.

OwnableInvalidOwner(address owner)

```

13  {
14      "inputs": [
15          {
16              "internalType": "address",
17              "name": "owner",
18              "type": "address"
19          }
20      ],
21      "name": "OwnableInvalidOwner",
22      "type": "error"
23  },

```

Parameter:

- **owner** (type address)

Functionality:

- Triggered when a user tries to transfer ownership to an invalid address
- The contract automatically reverts the transaction to avoid assigning ownership to a non-controllable or null address.

Security impact:

- Avoids misconfiguration by ensuring ownership is assigned only to valid addresses.

Events

OwnershipTransferred(address previousOwner, address newOwner)

```

35  {
36      "anonymous": false,
37      "inputs": [
38          {
39              "indexed": true,
40              "internalType": "address",
41              "name": "previousOwner",
42              "type": "address"
43          },
44          {
45              "indexed": true,
46              "internalType": "address",
47              "name": "newOwner",
48              "type": "address"
49          }
50      ],
51      "name": "OwnershipTransferred",
52      "type": "event"
53  },

```

You, 21 hours ago • Uncommitted changes

Parameters:

- **previousOwner** (type address): the address of the previous owner
- **newOwner** (type address): the address of the new owner

Functionality:

- Record every time the ownership of the contract is transferred from the old address to the new address.

Security impact:

Assignment 3

- Increase transparency in contract management and administration.
- Provides a clear audit trail of ownership changes.

Paused(address account)

```
54 {
55     "anonymous": false,
56     "inputs": [
57         {
58             "indexed": false,
59             "internalType": "address",
60             "name": "account",
61             "type": "address"
62         }
63     ],
64     "name": "Paused",
65     "type": "event"
66 },
```

Parameter:

- **account** (type address):

Functionality:

- Activates every time the contract switches to the “pause” state.
- Stops all the main functions (which will result in users not being able to perform certain events such as buying/selling products, creating new products, etc) in cases of emergency or maintenance (technical or operational issues, major contract upgrades)

Security impact:

- Supporting incidental or emergency maintenance.
- Help transparently change the important status of the contract.

Unpaused(address account)

```
107 {
108     "anonymous": false,
109     "inputs": [
110         {
111             "indexed": false,
112             "internalType": "address",
113             "name": "account",
114             "type": "address"
115         }
116     ],
117     "name": "Unpaused",
118     "type": "event"
119 },
```

Parameters: **account** (type address)

Functionality:

- Emitted when the contract is returned to normal operation (unpaused)

Security impact:

Assignment 3

- Signal restoration of functionality after being paused, allowing users to notice that the system is back online.

ProductCreated(uint256 productId, string name, uint256 price)

```
67 {
68     "anonymous": false,
69     "inputs": [
70         {
71             "indexed": true,
72             "internalType": "uint256",
73             "name": "productId",
74             "type": "uint256"
75         },
76         {
77             "indexed": false,
78             "internalType": "string",
79             "name": "name",
80             "type": "string"
81         },
82         {
83             "indexed": false,
84             "internalType": "uint256",
85             "name": "price",
86             "type": "uint256"
87         }
88     ],
89     "name": "ProductCreated",
90     "type": "event"
91 },
```

Parameters:

- **productId** (type uint256)
- **name** (type string)
- **price** (type uint256)

Functionality:

- Signals the creation of a valid product
- Validity is enforced via input checks in the **createProduct()** function.

Security impact:

- None.

ProductDeactivated(uint256 productId)

```

63 {
64     "anonymous": false,
65     "inputs": [
66     {
67         "indexed": true,
68         "internalType": "uint256",
69         "name": "productId",
70         "type": "uint256"
71     }
72     ],
73     "name": "ProductDeactivated",
74     "type": "event"
75 },

```

Parameters:

- **productId** (type uint256)

Functionality:

- Indicates a product has been manually deactivated, preventing further trade.

Security impact:

- Ensure that invalid products are not traded, thereby preventing risks and fraud.
- Make sure the product status changes are transparent.

ProductPurchased(uint256 purchaseId, uint256 productId, address buyer, uint256 amount)

```

76 {
77     "anonymous": false,
78     "inputs": [
79     {
80         "indexed": true,
81         "internalType": "uint256",
82         "name": "purchaseId",
83         "type": "uint256"
84     },
85     {
86         "indexed": true,
87         "internalType": "uint256",
88         "name": "productId",
89         "type": "uint256"
90     },
91     {
92         "indexed": true,
93         "internalType": "address",
94         "name": "buyer",
95         "type": "address"
96     },
97     {
98         "indexed": false,
99         "internalType": "uint256",
100        "name": "amount",
101        "type": "uint256"
102    }
103    ],
104    "name": "ProductPurchased",
105    "type": "event"
106 },

```

Parameters:

- **purchaseId** (type uint256)
- **productId** (type uint256)
- **buyer** (type address)
- **Amount** (type uint256)

Functionality:

- This event is used to signal a purchase of a product with ID **productID**, by the buyer with address **buyer** and amount **Amount**

Security impact:

- Provides traceability for transaction history and helps monitor suspicious payments.

Functions

createProduct

```
133 {
134   "inputs": [
135     {
136       "internalType": "string",
137       "name": "name",
138       "type": "string"
139     },
140     {
141       "internalType": "string",
142       "name": "description",
143       "type": "string"
144     },
145     {
146       "internalType": "uint256",
147       "name": "price",
148       "type": "uint256"
149     }
150   ],
151   "name": "createProduct",
152   "outputs": [],
153   "stateMutability": "nonpayable",
154   "type": "function"
155 }
```

Input:

- **name** (type string)
- **description** (type string)
- **price** (type uint256 - wei)

Output: Emits a **ProductCreated** event

Functionality:

- Check the requirements: **price** is larger than 0; **name** and **description** must be non-empty. If successful, then proceed.
- Saving the product into the **products** mapping(uint256 => Product)

Assignment 3

- Emit the **ProductCreated** event

Security:

- It does not have onlyOwner, so everyone can create products in the marketplace

Purpose:

- Everyone can post and sell their products on the marketplace.
- Use **whenNotPaused** to prevent operations when the contract is paused.
- Requiring validated input (**price**, **name**, **description**) to prevent spam.

buyProduct

```
120 {
121   "inputs": [
122     {
123       "internalType": "uint256",
124       "name": "productId",
125       "type": "uint256"
126     }
127   ],
128   "name": "buyProduct",
129   "outputs": [],
130   "stateMutability": "payable",
131   "type": "function"
132 }
```

Input: **productId** (type uint256)

Output: Emits a **ProductPurchased** event if successful, no output otherwise

Functionality:

- Checking the product if they are still active
- Create a new **Purchase** with a new ID, save to the mapping (uint256 => Purchase) public purchases; and the array **userPurchases[msg.sender]**
- Transfer ETH for the seller, refund the residue to the buyer
- Emit the **ProductPurchased** event

Security impact:

- nonReentrant: preventing reentrancy attack

Purpose:

- Support residual refund, avoiding buyers' lost ETH excess.
- When the contract is paused, the executions will be blocked.

pause

```
{
  "inputs": [],
  "name": "pause",
```

Assignment 3

```
    "outputs": [],  
    "stateMutability": "nonpayable",  
    "type": "function"  
  },
```

Input and output: none

Functionality:

- Temporarily disables contract functionality by setting the paused state to true.
- prevents specific functions, such as buyProduct, from executing while paused.

Security impact:

- Can only be called by the contract owner.
- Helps prevent misuse or vulnerabilities during abnormal situations.

unpause

```
{  
  "inputs": [],  
  "name": "unpause",  
  "outputs": [],  
  "stateMutability": "nonpayable",  
  "type": "function"  
},
```

Parameters: None (no input and output)

Functionality:

- Call `_unpause()` to reopen the contract

Security Impact:

- Only the owner can do that, so it can restore activities after fixing issues.

deactivateProduct

```
156 {  
157   "inputs": [  
158     {  
159       "internalType": "uint256",  
160       "name": "productId",  
161       "type": "uint256"  
162     }  
163   ],  
164   "name": "deactivateProduct",  
165   "outputs": [],  
166   "stateMutability": "nonpayable",  
167   "type": "function"  
168 },
```

Input: `productId` (type uint256)

Functionality:

- Deactivate the product with the given ID by setting `products[productId].active = false` and emitting the event `ProductDeactivated(productId)`

Security Impact:

- Only the owner can do that, protect the integrity of the Marketplace

renounceOwnership

```
331 {
332   "inputs": [],
333   "name": "renounceOwnership",
334   "outputs": [],
335   "stateMutability": "nonpayable",
336   "type": "function"
337 },
```

Input and output: none

Functionality:

- Allows the current owner to cancel the ownership, setting the owner to zero address, which makes the property ownerless

Security impact:

- Can not be reverted
- After renouncement, privileged owner actions are no longer possible.

transferOwnership

```
338 {
339   "inputs": [
340     {
341       "internalType": "address",
342       "name": "newOwner",
343       "type": "address"
344     }
345   ],
346   "name": "transferOwnership",
347   "outputs": [],
348   "stateMutability": "nonpayable",
349   "type": "function"
350 },
```

Input: `newOwner` (type address)

Output: `[]` (none, empty list / array)

Functionality:

- Transfers ownership of the contract to a new address, ensuring proper administrative control and emitting an `OwnershipTransferred` event.

Security:

- Only the owner can call, maintaining control
- Allow the current contract owner to hand over full control to another address

getPurchases

```
{
  "inputs": [
    {
      "internalType": "address",
      "name": "user",
      "type": "address"
    }
  ],
  "name": "getPurchases",
  "outputs": [
    {
      "components": [
        {
          "internalType": "uint256",
          "name": "purchaseId",
          "type": "uint256"
        },
        {
          "internalType": "uint256",
          "name": "productId",
          "type": "uint256"
        },
        {
          "internalType": "address",
          "name": "buyer",
          "type": "address"
        },
        {
          "internalType": "uint256",
          "name": "timestamp",
          "type": "uint256"
        },
        {
          "internalType": "uint256",
          "name": "amount",
          "type": "uint256"
        }
      ]
    }
  ]
}
```

Assignment 3

```
    ],  
    "internalType": "struct Marketplace.Purchase[]",  
    "name": "",  
    "type": "tuple[]"  
  }  
],  
"stateMutability": "view",  
"type": "function"  
},  
},
```

Input: **user** (type address)

Output: an array **Purchase[]** of purchases made by the user with ID **user**

Functionality:

- Read the **userPurchases[user]**, build and return the whole transaction of that user

Security impact: none

Purpose:

- Displays the purchase history

purchase(uint256 purchaseId)

```
{  
  "inputs": [  
    {  
      "internalType": "uint256",  
      "name": "",  
      "type": "uint256"  
    }  
  ],  
  "name": "purchases",  
  "outputs": [  
    {  
      "internalType": "uint256",  
      "name": "purchaseId",  
      "type": "uint256"  
    },  
    {  
      "internalType": "uint256",  
      "name": "productId",  
      "type": "uint256"  
    }  
  ],  
  {  
    "internalType": "uint256",  
    "name": "",  
    "type": "uint256"  
  }  
}
```

Assignment 3

```
        "internalType": "address",
        "name": "buyer",
        "type": "address"
    },
    {
        "internalType": "uint256",
        "name": "timestamp",
        "type": "uint256"
    },
    {
        "internalType": "uint256",
        "name": "amount",
        "type": "uint256"
    }
],
"stateMutability": "view",
"type": "function"
},
```

Parameter:

- **purchaseId** (type uint256)

Output:

- Returns a Purchase struct including **purchaseId** (uint256), **productId** (uint256), **buyer** (address), **timestamp** (uint256), **amount** (uint256)

Functionality:

- Retrieves a purchase record directly by its unique ID from the global purchases mapping.

Security impact:

- Read-only and safe unless sensitive private data were involved

Purpose:

- Supports auditing, analytics, and frontend UI display of purchase details.

paused

```

264     {
265         "inputs": [],
266         "name": "paused",
267         "outputs": [
268             {
269                 "internalType": "bool",
270                 "name": "",
271                 "type": "bool"
272             }
273         ],
274         "stateMutability": "view",
275         "type": "function"
276     },

```

Parameters: None (no input and output)

Functionality:

- Set the contract status to “paused” via the `_pause()` function in the OpenZeppelin Pausable contracts.

Security Impact: None

userPurchases(address user, uint256 index)

```

387     {
388         "inputs": [
389             {
390                 "internalType": "address",
391                 "name": "",
392                 "type": "address"
393             },
394             {
395                 "internalType": "uint256",
396                 "name": "",
397                 "type": "uint256"
398             }
399         ],
400         "name": "userPurchases",
401         "outputs": [
402             {
403                 "internalType": "uint256",
404                 "name": "",
405                 "type": "uint256"
406             }
407         ],
408         "stateMutability": "view",
409         "type": "function"
410     }
411 ]

```

Parameters:

- `user` (type address)
- `index` (type uint256)

Output:

- `purchaseId` (type uint256)

Functionality:

Returns the purchase ID at a given index from the array `userPurchases[user]`

- Helps applications iterate through all purchases made by a user, one by one.

Security impact:

- Read-only and safe

Purpose:

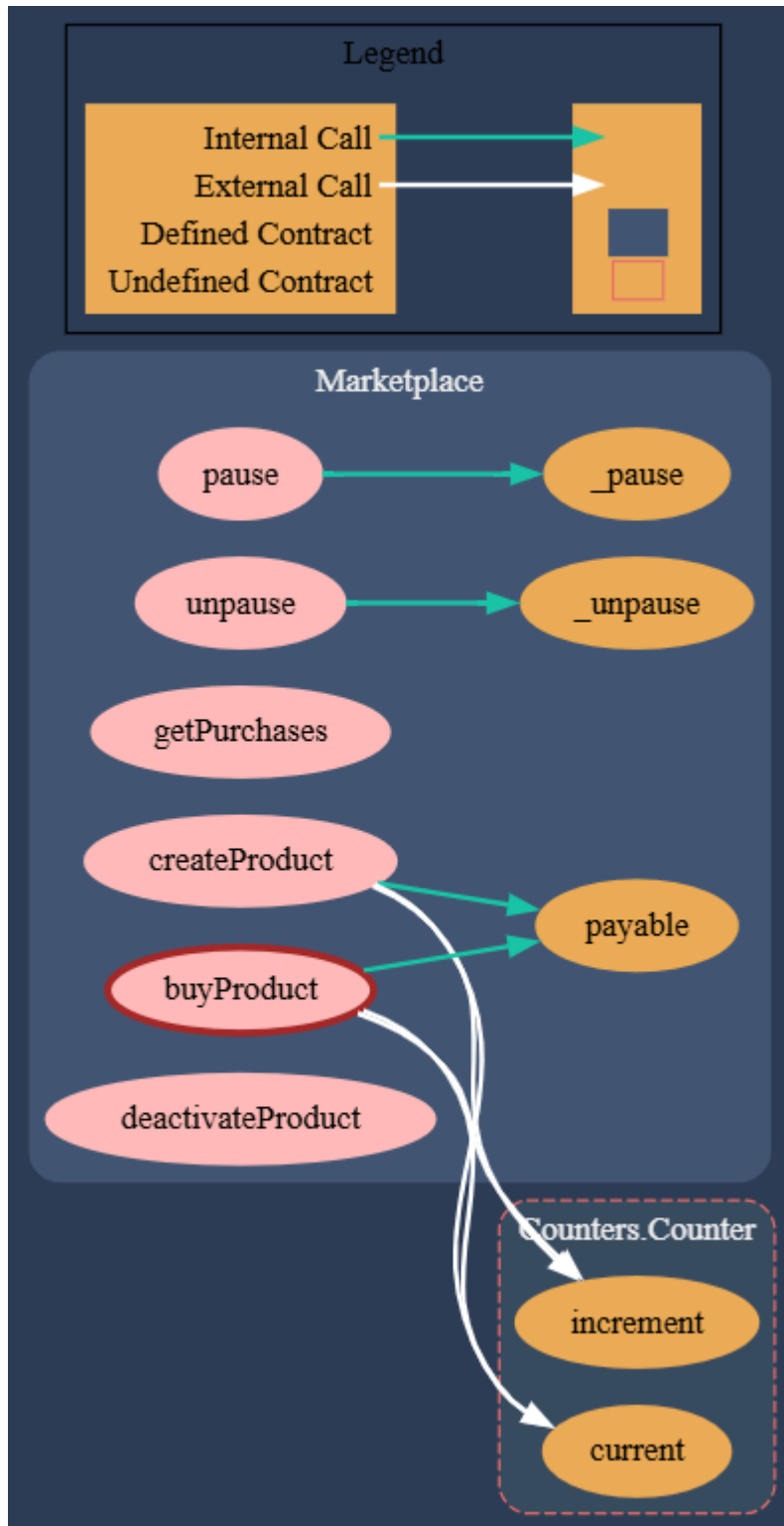
- Serves as a lightweight method for off-chain systems to fetch a user's purchase history incrementally without loading the entire array at once.

Call Graph Discussion

Overview:

The call graph of the Marketplace smart contract provides a comprehensive understanding of the interactions, communications, and dependencies between its various functions. Along with the inherited functions, this graph also shows the external and internal dependencies that form the contract's main logical flow.

The call graph contains a brief of the functions used in the contract, as well as the calling logic between them. In the graph, there are two types of arrows: Teal and White. The teal arrows represent internal calls, which happen when a function in a contract calls another one in the same contract. The white arrow represents external calls, which are calls from a function in a contract to another function in a different contract. For example, `buyProduct -> payable` is an internal (teal) call, `buyProduct -> Counter.increment` is an external (white) call.



The graph was generated using the Solidity Visual Developer extension in Visual Studio Code.

Core Functional Groups:

Admin/Owner controls:

Assignment 3

- `_pause` and `unpause`
- Activate or deactivate the contract for maintenance or having incidents.

Product Management:

- `createProduct`, `deactivateProduct`
- Create or make the existing product unavailable

Transaction Processing

- `buyProduct`
- Executes the payment and updates data.

User Data Access:

- `getPurchase`
- Displays transaction history for the user

Function classification and detailed overview

Custom Defined Functions: defined for the assignment.

- **`createProduct(string name, string description, uint256 price)`**: Calls `increment()` to increase the product ID counter, then retrieves the ID using `current()`. It also accepts the purchase via payable and stores data on-chain.
- **`buyProduct(uint256 productId)`**: This function handles the purchasing process, using `current()` to check the product ID. Using `nonReentrant` for security. The `buyProduct()` function interacts with `getPurchases()` to update the transaction history.
- **`deactivateProduct (uint256 productId)`**: Deactivate the product by setting its status to inactive.
- **`getPurchase(address user)`**: Retrieving payment records, interacts with `buyProduct()` to either read or write user purchase data.
- **`pause()`**: calls `_pause()` to disable contract actions.
- **`unpause()`**: calls `_unpause()`, re-enable contract actions, opposite of `pause()`. These are owner-only functions that trigger inherited internal calls from `Pausable` to activate or deactivate the contract.

Modifiers:

- **`onlyOwner`**: For owners and admins only, allows calling `pause()` and `unpause()` for the contract. Restricts `pause()`, `unpause()`, and `deactivateProduct()` to contract owner.
- **`whenNotPaused`**: Ensures that certain functions such as `buyProduct()`, can only be executed when the contract is active.
- **`nonReentrant`**: Protects `buyProduct()` against reentrancy attacks.

Inherited functions: inherited from the OpenZeppelin v4.9.6 library

- `_pause()` and `unpause()` from `Pausable`: the internal functions used to change the contract's operational status.

- `_increment()` and `current()` from `Counters`: used to generate and get the product ID safely without duplication.
- `transfer()` and `payable()` from Solidity: Handle ETH transfers.

Detailed Inter-Function Calling Logic:

`createProduct(string name, string description, uint256 price)`

This function enables the user to post their created product on the shop. It verifies input validity, such as name, description and price, and generates a unique ID for each product.

- Validates that name, description, and price are not empty or zero.
- Calls `Counter.increment()` to increase the product ID counter.
- Calls `Counter.current()` to retrieve the new product ID.
- Creates and stores a new `Products` struct in the product mapping.
- Emits `ProductCreated` event with relevant metadata.

`buyProduct(uint256 productId)`

Handles the core purchasing logic and transfers ETH to the seller. Uses `nonReentrant` for security.

- Fetches the product from `product[productId]`.
- Checks if the product is active and if the buyer sent enough ETH.
- Calls `Counter.increment()` to assign a unique purchase ID.
- Stores the purchase in the purchase mapping.
- Appends the purchase ID to `userPurchases[msg.sender]`.
- Sets the product's active status to false.
- Transfers ETH to the seller.
- Refund any overpayment to the buyer.
- Emits `ProductPurchased`.

`deactivateProduct(uint256 productId)`

Admin-only function that deactivates a product manually

- Validates if the product is active
- Set `active = false` in the product struct.
- Emits `ProductDeactivated`.

`getPurchases(address user)`

A read-only function that lets user check their transaction history

- Look up the list of purchase IDs tied to the user's address
- Goes through each ID and finds the matching purchase information
- Returns all the purchase details in one array

Security and Modifiers:

- **`onlyOwner`**: assigns that only the owner can operate `pause/unpause/deactivateProduct`, and restricts the right to call these functions.

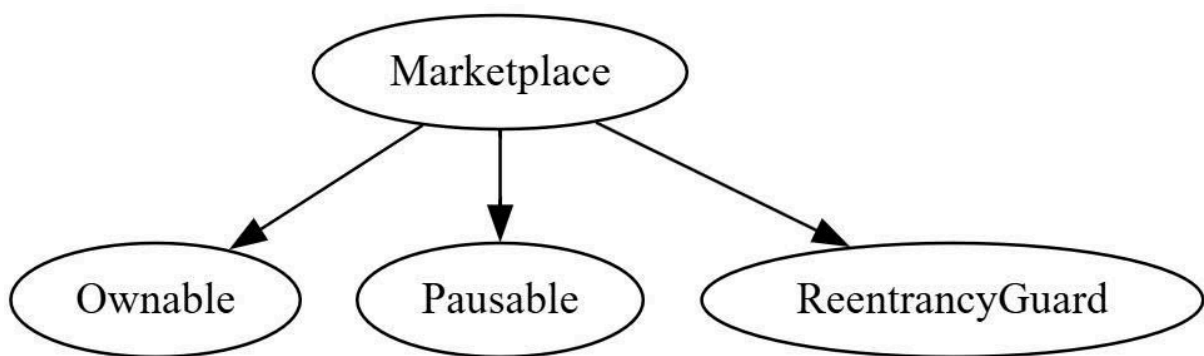
- **whenNotPaused**: ensures that buyProduct and createProduct can only be active when the contract is active.
- **nonReentrant**: protects the buyProduct function from reentrancy vulnerabilities by preventing nested calls.

Event Handling and Off-chain Interaction:

- **ProductCreated(uint256 productId, string name, uint256 price)**
Triggered when a new product is listed.
- **ProductPurchased(uint256 purchaseId, uint256 productId, address buyer, uint256 amount)**
Triggered after a successful purchase is processed.
- **ProductDeactivated(uint256 productId)**
Triggered when a product is manually deactivated by the owner.

The function logic in Marketplace is well-structured and clearly separated by responsibility. Each core function performs its task with appropriate validations, event logging, and library usage—supporting both usability and security.

Inheritance graph discussion



Inheritance relationships:

The Marketplace.sol contract demonstrates an inheritance structure by extending functions from OpenZeppelin contracts, such as Ownable, Pausable, and ReentrancyGuard. As documented in Solidity: “Solidity supports multiple inheritance, and regardless of how many base contracts a derived contract inherits from, only one contract instance is actually deployed on-chain, combining the flattened code from all the base contracts.” (Solidity Programming, March 2017, pp. 69–88). The Ownable contract is used to implement

ownership access restrictions and to have ownership functions. Integration with Pausable gives the emergency stop capabilities, allowing contract functions to be stopped or resumed through controlled triggers. Additionally, ReentrancyGuard provides protection against reentrancy attacks with the transaction functions.

The implementation reduces the contract's complexity while enhancing its reliability and security.

Code Reusability:

The inheritance graph inherits three base contracts from the OpenZeppelin library simultaneously.

- Owner (managing ownership): provides owner() to take the owner's address. onlyOwner indicates that only the owner has the right to call the function and the transferOwnership() function.
- Pausable (circuit-braker, pause): including the paused, _pause() and _unpause(), and whenNotPaused modifier
- ReentrancyGuard: Implements the nonReentrant modifier, used to block recursive calls in sensitive functions like buyProduct.

In Object-Oriented Programming, “inheritance is a way to compartmentalise and reuse code by creating collections of attributes and behaviours called objects that can be based on previously created objects” (G Goyal, S Patel, 2012). Solidity also borrows this definition to organise the contract logic. Additionally, there is less chance of creating errors in fundamental logic when using well-maintained libraries like OpenZeppelin.

Function and State Variable Overriding:

We haven't implemented this feature, but we have plans in the future to expand this project further and implement it.

Function Visibility and Access Control

Admin functions like pause, upause, or deactivateProduct are restricted with onlyOwner to make sure only the admin can use them. User functions like createProduct and buyProduct are external and protected by whenNotPaused and nonReentrant, to help prevent misuse and reentrancy attacks. Visibility for variables and mappings is set to balance access and security.

Modifier Application

Modifier application is how the modifiers can be inherited and applied to the functions to alter the behaviour and enhance security.

onlyOwner (inherited): only allow the owner to call these functions, preventing unauthorised users from managing operations such as pausing the contract or deactivating products.

- nonReentrant (inherited)

Assignment 3

- `whenNotPaused`: Stop the execution when the contract is paused. The owner can pause the create/buy product process if there is an abnormality

For example, `buyProduct` has both `nonReentrant` and `whenNotPaused`

```
function buyProduct(uint256 productId) external payable nonReentrant
whenNotPaused {
```

It means that the purchase process can only continue when the contract is unpaused and...

Or `deactivateProduct` has `onlyOwner`

```
function deactivateProduct(uint256 productId) external onlyOwner {
```

So only the owner can deactivate the products.

By inheriting and applying modifiers, the contract builds a layer quite similar to middleware: core function logic stays focused on the behaviour, meanwhile, the status and requirements are separated through modifiers. This concept reduces the risk of security vulnerability.

Conclusion

In summary, the Marketplace smart contract utilised in the Happy Farming project has been thoroughly examined in this paper. Using OpenZeppelin's `Ownable`, `Pausable`, and `ReentrancyGuard` modules, the solution effectively combines modular design, access control, and safe transaction methods. Effective use of inheritance and modifiers was made to provide usability and defence against common weaknesses. In order to make the contract more useful in actual decentralised markets, it may be expanded in the future to include features like product ratings, role-based permissions, or multi-user listing capabilities.

References:

OpenZeppelin (2024). Access Control - Ownable, OpenZeppelin Documentation, accessed 4 August 2025. <https://docs.openzeppelin.com/contracts/4.x/access-control#ownable>

OpenZeppelin (2024) Pausable, OpenZeppelin Documentation, accessed 4 August 2025. <https://docs.openzeppelin.com/contracts/4.x/api/security#pausable>

OpenZeppelin (2024) ReentrancyGuard, OpenZeppelin Documentation, accessed 4 August 2025. <https://docs.openzeppelin.com/contracts/4.x/api/security#reentrancyguard>

Solidity (2024). Solidity by Example – Inheritance, Solidity Docs, accessed 4 August 2025. <https://solidity-by-example.org/inheritance/>

Dannen, C. (2017). Introducing Ethereum and Solidity: Foundations of Cryptocurrency and Blockchain Programming for Beginners, 1st edn, Berkeley, CA: Apress. <https://doi.org/10.1007/978-1-4842-2535-6>